



SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)

Constituent of Symbiosis International (Deemed University), Pune

(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of India)
Re-Accredited by NAAC with 'A++' Grade

Name: Aashray Lal kayastha

PRN: 25070122005

Division: A1

ASSIGNMENT No: 10

TITLE: Write a Java program to demonstrate the use of abstract classes and abstract methods

Problem Statement

Write a Java program to demonstrate the use of abstract classes and abstract methods

Learning Objectives

To implement abstraction using abstract classes and abstract methods.

Theory

Abstract classes provide partial implementation of a program and cannot be instantiated directly. They may contain both abstract and concrete methods. Abstract methods declare functionality that must be implemented by subclasses. Abstraction hides implementation details while exposing only essential functionality. This improves code flexibility and maintainability in object-oriented programming.

Outcome:

Abstract classes and abstract methods were successfully used to define a common structure while leaving implementation details to subclasses, as verified through the sample programs — each subclass implemented the abstract method in its own way.

```
abstract class Shape{
    String color;

    Shape(String c){
        this.color = c;
    }

    abstract double area();

    void showColor(){
        System.out.println("Color: " + color);
    }
}

class Circle extends Shape{
    double radius;

    Circle(String c, double r){
        super(c);
        this.radius = r;
    }

    double area(){
        return Math.PI * radius * radius;
    }
}

class Rectangle extends Shape{
    double length, width;

    Rectangle(String c, double l, double w){
        super(c);
        this.length = l;
        this.width = w;
    }

    double area(){
        return length * width;
    }
}

public class JAVA_EXP_10{
    public static void main(String [] args){
```

```
Shape circle = new Circle("Red",5);
circle.showColor();
System.out.println("Circle Area: " + circle.area());

System.out.println();

Shape rectangle = new Rectangle("Blue",4,6);
rectangle.showColor();
System.out.println("Rectanle Area: " + rectangle.area());
}
```

Output:

```
C:\Users\Aashray\OneDrive\Desktop\JAVA_PRACTICALS>java JAVA_EXP_10.java
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Color: Red
Circle Area: 78.53981633974483

Color: Blue
Rectanle Area: 24.0
```

Conclusion:

Therefore, from the above experiment we got to learn the use of abstract classes and abstract methods

Exercise

1. Create an abstract Payment class and implement Credit Card and UPI payment classes.?

Source Code:

```
1  abstract class Payment {
2
3      abstract void makePayment(double amount);
4  }
5
6  class CreditCard extends Payment {
7
8      void makePayment(double amount) {
9          System.out.println(x: "Payment made using Credit Card.");
10         System.out.println("Amount: ₹" + amount);
11     }
12 }
13
14 class UPI extends Payment {
15
16     void makePayment(double amount) {
17         System.out.println(x: "Payment made using UPI.");
18         System.out.println("Amount: ₹" + amount);
19     }
20 }
21
22 public class assignment10_PaymentDemo {
23     Run | Debug | Run main | Debug main
24     public static void main(String[] args) {
25
26         Payment p1 = new CreditCard();
27         p1.makePayment(amount: 2500);
28
29         System.out.println();
30
31         Payment p2 = new UPI();
32         p2.makePayment(amount: 1200);
33     }
34 }
```

Output:

```
C:\Users\Aashray\Documents\University\3rd sem\JAVA>java assignment10_PaymentDemo.java
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Payment made using Credit Card.
Amount: ₹2500.0

Payment made using UPI.
Amount: ₹1200.0
```

2. Create an abstract FoodOrder class with an abstract method calculateBill(). Implement two subclasses, DineInOrder and TakeAwayOrder, to calculate and display the total bill based on the order type.

Source Code:

```
1  abstract class FoodOrder {
2
3      double amount;
4
5      FoodOrder(double amount) {
6          this.amount = amount;
7      }
8
9      abstract void calculateBill();
10 }
11
12 class DineInOrder extends FoodOrder {
13
14     DineInOrder(double amount) {
15         super(amount);
16     }
17
18     void calculateBill() {
19         double total = amount + (amount * 0.10); // 10% service charge
20         System.out.println(x: "Dine-In Order");
21         System.out.println("Food Amount : ₹" + amount);
22         System.out.println("Total Bill : ₹" + total);
23     }
24 }
25
26 class TakeAwayOrder extends FoodOrder {
27
28     TakeAwayOrder(double amount) {
29         super(amount);
30     }
31
32     void calculateBill() {
33         double total = amount + 30; // ₹30 packing charge
34         System.out.println(x: "Take Away Order");
35         System.out.println("Food Amount : ₹" + amount);
36         System.out.println("Total Bill : ₹" + total);
37     }
38 }
39
```

```
40 public class assignment10_FoodOrderDemo {  
    Run | Debug | Run main | Debug main  
41     public static void main(String[] args) {  
42  
43         FoodOrder order1 = new DineInOrder(amount: 500);  
44         order1.calculateBill();  
45  
46         System.out.println();  
47  
48         FoodOrder order2 = new TakeAwayOrder(amount: 500);  
49         order2.calculateBill();  
50     }  
51 }
```

Output:

```
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8  
Dine-In Order  
Food Amount : ₹500.0  
Total Bill  : ₹550.0  
  
Take Away Order  
Food Amount : ₹500.0  
Total Bill  : ₹530.0
```